TRƯỜNG ĐẠI HỌC FPT TP.HCM

KHOA VI MẠCH BÁN DẪN – ÔTÔ SỐ

—— o0o ——



FPT UNIVERSITY

BÁO CÁO ĐỒ ÁN

HỆ THỐNG KHÓA CỬA NHẬN DIỆN KHUÔN MẶT

GVHD:	TS. Lê Thế Dũng
Lớp:	CDP301 – IC2004
Nhóm:	2
Trần Nguyễn Việt Dũng	SE205180
Trần Gia Phúc	SE204417
Trần Hoàng Khôi	SE203155
Thân Trọng Khang	SE203290
Nguyễn Huy Khánh	SE204207

TP. Hồ Chí Minh, ngày 15 tháng 06 năm 2026

LECTURER'S REVIEW

Project Title: Face Recognition Door Lock System

Group: 2

Students: Tran Nguyen Viet Dung, Tran Gia Phuc, Tran Hoang Khoi, Than Trong Khang, Nguyen Huy Khanh

Instructor: Dr. Le The Dung

Comments from Instructor:

Final Score: _____ / 10

Ho Chi Minh City, _____ 2026

Instructor's Signature

Dr. Le The Dung

MEMBER CONTRIBUTIONS

The overall execution of this project was divided systematically among all team members. Table 0.1 summarizes the primary responsibilities and workload contribution of each member.

No.	Student Name	Assigned Tasks / Primary Responsibilities	Contribution
1	Trần Nguyễn Việt Dũng	Hardware Assembly, & Technical Report Writing, Flowchart & System Block Diagram Documentation.	20%
2	Trần Gia Phúc	Firmware Development (ESP32-CAM), Circuit Schematic & 2-Layer PCB Layout Design.	20%
3	Trần Hoàng Khôi	Hardware Assembly, Enclosure Construction, Power Management & Relay Driving Circuit Testing.	20%
4	Thân Trọng Khang	Testing & Experimental Data Collection, Hardware Verification.	20%
5	Nguyễn Huy Khánh	System Architecture Design, AI Server API Integration, Web Dashboard Development.	20%
		Total	**100%**

Table 0.1: Task Assignment and Workload Contribution Breakdown..

Group Leader's Confirmation:
We hereby confirm that all team members have completed their assigned tasks with high responsibility and dedication.

Group Leader

Trần Nguyễn Việt Dũng

ABSTRACT

In the context of modern residential security, traditional access control methods such as physical keys and RFID cards are increasingly vulnerable to loss, theft, and duplication. To address these critical security flaws, this project aims to develop a smart, automated door lock system utilizing facial recognition technology. The methodology integrates edge hardware with cloud computing, employing the ESP32-CAM microcontroller for image acquisition and environmental sensing via a Light Dependent Resistor (LDR) for automated low-light compensation. The captured visual data is securely transmitted to a third-party server (Face++ API) which utilizes advanced Deep Feature Extraction algorithms to verify authorized users. Upon successful authentication, the microcontroller triggers a 5V relay to actuate a 12V solenoid door lock. Experimental results demonstrate that the system operates with high stability, achieving a high recognition accuracy rate. Furthermore, the system maintains a consistent end-to-end processing latency between 1.8 and 2.1 seconds, successfully validating the integration of IoT hardware and cloud-based biometric authentication for secure, hands-free access control.

Table of contents

List of Figures **6**

1 INTRODUCTION **7**

1.1 Research Tasks and Scope 8

1.1.1 Technical Objectives 8

1.1.2 Implementation Content 8

2 SYSTEM DESIGN AND IMPLEMENTATION **9**

2.1 System Overview 9

2.2 System Block Diagram 9

2.3 Hardware Components Detail 10

2.3.1 ESP32-CAM Microcontroller 10

2.3.2 LM2596S DC-DC Buck Converter Module 11

2.3.3 12V Solenoid Door Lock 11

2.3.4 5V Relay Module and 2N2222 NPN Transistor 12

2.3.5 I2C LCD 2004 Display 12

2.3.6 Light Dependent Resistor (LDR) and Illumination LED . . 13

2.4 Hardware Design 13

2.4.1 Schematic Design 13

2.4.2 Printed Circuit Board (PCB) Design 15

2.5 Software Design and Algorithms 16

2.5.1 System Operation Flowchart 16

2.5.2 Facial Recognition Algorithm 17

3 SYSTEM TESTING AND RESULTS **24**

3.1 Real-Life Deployment 24

3.2 Hardware Prototype 24

3.3 Functional Hardware Verification 25

3.3.1 Web-Based Monitoring and History Log 27

3.4 Experimental Results and Accuracy 28

3.4.1 Limitations and Analysis 28

3.4.2 Future Improvements 30

3.4.3 API Latency Analysis and Network Jitter 30
3.4.4 System Reliability and Asynchronous Execution Observations 31

4 CONCLUSION **33**
4.1 Summary of Achievements 33
4.2 System Limitations 33
4.3 Future Works 33

References **34**

APPENDIX **35**

A Wiring Connection and Pinout Table **35**

B Environment Setup and Flashing Firmware **36**

C ESP32-CAM Source Code (Firmware Snippet) **37**

D Project Repository (Open Source) **39**

List of Figures

2.1 Overall System Block Diagram 10
2.2 ESP32-CAM Module Pinout Diagram 10
2.3 LM2596S DC-DC Step-Down Buck Converter 11
2.4 12V DC Solenoid Door Lock Mechanism 11
2.5 Power Switching and Isolation Components 12
2.6 LCD 2004 with PCF8574 I2C Interface Backpack 12
2.7 Low-Light Compensation Components 13
2.8 Overall System Schematic Diagram 14
2.9 Component Placement and Routing (2D PCB Layout) 16
2.10 3D PCB Model Perspectives 16
2.11 System Firmware and Operation Flowchart 17
3.1 Prototype Deployment of the Face Recognition Door Lock System 24
3.2 Assembled PCB 25
3.3 Test Case 1: Initial Standby State 25
3.4 Test Case 2: LDR Covered for Image Capture (LED On) 26
3.5 Test Case 3: Solenoid Retracted (Unlocked) 26
3.6 Log Page 1 27
3.7 Log Page 2 28
3.8 Comparison of Total vs. Successful Identifications 29
3.9 Breakdown of System Outcomes across 62 Total Trials 30

1. INTRODUCTION

As the integration of artificial intelligence into daily life becomes increasingly prevalent, its applications extend beyond mere convenience to significantly enhance residential security [5][1]. In this context, the front door serves as the critical first line of defense against potential threats.

Up to now, we have relied on traditional methods like physical keys, PIN codes, or RFID cards. But these all have their flaws: keys can be lost or copied, cards can be stolen, and passwords are easy to forget or hack. Because of these hassles, we need something smarter, safer, and entirely hands-free way to unlock our doors.

Biometric technology, specifically facial recognition, offers a robust solution to these vulnerabilities by providing a contactless, highly efficient, and user-friendly alternative [7][2]. Motivated by the rapid evolution of human-computer interaction, this research focuses on the design and development of a **"Face Recognition Door Lock System"**. Our primary objective is to engineer a functional prototype capable of executing the following core tasks:

- Real-time Image Acquisition: Capturing live user imagery via an ESP32-CAM module and securely transmitting the data to a centralized server for processing.
- Algorithmic Verification: Implementing advanced computer vision algorithms to accurately detect and verify authorized individuals while effectively denying access to unrecognized entities.
- Automated Actuation: Triggering the physical unlocking mechanism seamlessly upon successful authentication, ensuring a secure and hands-free entry process.

Ultimately, this project presents an invaluable opportunity to bridge the gap between software algorithms and physical hardware. By translating theoretical principles

[1] M. A. Al-Qurabat and A. K. Idrees, “Smart Home Security System Based on Internet of Things,” 2020.

[2] K. K. Patel and S. M. Patel, “Internet of Things-IOT: Definition, Characteristics, Architecture, Enabling Technologies, Application & Future Challenges,” 2016.

into a practical, real-world application, we aim to cultivate comprehensive hands-on research experience in embedded systems and automated security.

1.1 Research Tasks and Scope

1.1.1 Technical Objectives

The core objective of this project is to research, design, and implement a complete embedded smart door lock system, utilizing the ESP32-CAM microcontroller as the central processing unit. The system must satisfy strict technical criteria regarding real-time processing constraints in computer vision algorithms for facial recognition and feature extraction. Furthermore, the hardware architecture must ensure high reliability in its electromechanical execution block, maintaining a Fail-Secure state, and providing seamless environmental adaptability through automated low-light compensation mechanisms.

1.1.2 Implementation Content

The project implementation process is systematically broken down into three consecutive and interdependent phases. The initial phase focuses on theoretical research and component selection, including an in-depth analysis of technical specifications (Datasheets) to optimize compatibility between hardware elements and ensure safe isolation. The second phase involves hardware design and simulation, encompassing the calculation of the schematic diagram and the routing of a double-sided Printed Circuit Board (2-layer PCB), with a primary emphasis on ensuring digital signal integrity and electromagnetic compatibility (EMC). The final phase is dedicated to firmware and software development, concentrating on structuring the embedded source code to manage multi-tasking data streams, integrating the facial recognition pipeline, and optimizing peripheral feedback control for user interaction.

2. SYSTEM DESIGN AND IMPLEMENTATION

2.1 System Overview

The Face Recognition Door Lock System operates on an edge-to-cloud IoT architecture, seamlessly integrating local hardware peripherals with cloud-based computer vision algorithms [6][3]. The ESP32-CAM microcontroller serves as the central edge node, responsible for real-time environmental sensing and image acquisition. To adapt to varying environmental factors, an integrated Light Dependent Resistor (LDR) continuously monitors ambient illumination, automatically triggering a high-intensity white LED to ensure optimal image quality under low-light conditions.

Upon image capture, the software integration takes place as the ESP32-CAM packages the visual payload and transmits it via Wi-Fi to a dedicated AI server using an API. The server-side algorithm executes facial detection, feature extraction, and database matching to determine user authorization. Once the verification result is transmitted back to the microcontroller, the ESP32-CAM translates this digital response into physical actuation. It provides immediate user feedback via an I2C LCD and an active buzzer. If access is granted, the microcontroller outputs a control signal to a 2N2222 transistor, which energizes a 5V relay to retract the 12V solenoid door mechanism. This synchronized interaction between edge hardware and cloud-based AI forms a secure, highly automated, and hands-free access control system.

2.2 System Block Diagram

The system is powered by a 9V DC supply. A step-down converter reduces the voltage to 5V to power the ESP32-CAM. The ESP32-CAM captures an image and transmits it to the AI server via Wi-Fi for facial recognition. If the detected face matches one of the authorized faces stored in the database, the server sends a recognition result back to the ESP32-CAM. The ESP32-CAM then activates the relay module, which unlocks the solenoid lock.

[3] A. K. Gupta, et al., "Smart Door Lock System using ESP32 CAM," *IJERT*, 2021.

The overall system architecture is depicted in the block diagram below. It illustrates the functional relationship between the input sensors, the central processing unit, and the output actuators. A detailed technical breakdown of each specific hardware component presented in this diagram is extensively discussed in Section 2.3.

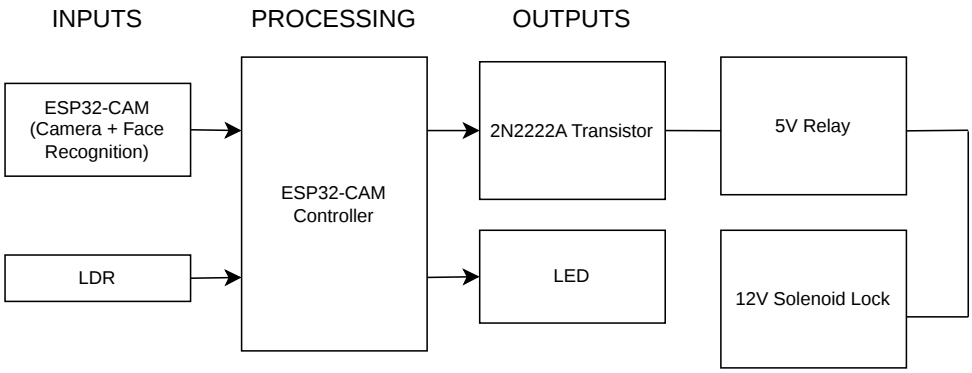


Figure 2.1: Overall System Block Diagram.

2.3 Hardware Components Detail

2.3.1 ESP32-CAM Microcontroller

The visual data acquisition core of the system is built upon the ESP32-CAM module platform. Unlike standard microcontrollers, this is a System-on-Chip (SoC) specifically optimized for IoT applications requiring complex image processing. It features a Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor operating at a maximum clock speed of 240MHz.



Figure 2.2: ESP32-CAM Module Pinout Diagram.

2.3.2 LM2596S DC-DC Buck Converter Module

The electromagnetic door lock system requires a primary 12V DC voltage, whereas the logic circuit operates at 5V and 3.3V. To resolve this discrepancy, the LM2596 DC-DC Buck Converter module is utilized. Operating on a Pulse Width Modulation (PWM) mechanism with an internal switching frequency of 150kHz, it steps down the 12V input to a flat, stable 5V DC level.

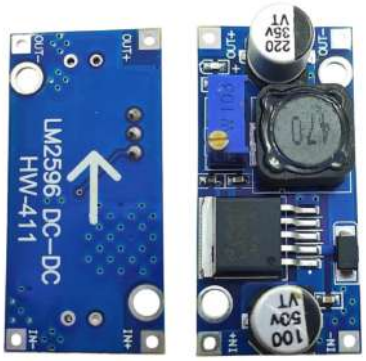


Figure 2.3: LM2596S DC-DC Step-Down Buck Converter.

2.3.3 12V Solenoid Door Lock

The primary actuator of the system is the Solenoid Door Lock, operating at a nominal rating of 12V DC. The lock functions based on electromagnetic induction. When a high-intensity 12V current flows through its internal copper coil, it generates a strong magnetic force that retracts the steel slug, releasing the latch.



Figure 2.4: 12V DC Solenoid Door Lock Mechanism.

2.3.4 5V Relay Module and 2N2222 NPN Transistor

To safely bridge the low-voltage logic of the ESP32-CAM (3.3V) and the 12V high-power solenoid lock, a 5V Electromechanical Relay is implemented. However, the ESP32-CAM's GPIO pins cannot source enough current to directly energize the relay coil. Therefore, a 2N2222 NPN Bipolar Junction Transistor (BJT) is employed as a low-side switch.



(a) 1-Channel 5V Relay Module.



(b) 2N2222 NPN Transistor Pinout.

Figure 2.5: Power Switching and Isolation Components.

2.3.5 I2C LCD 2004 Display

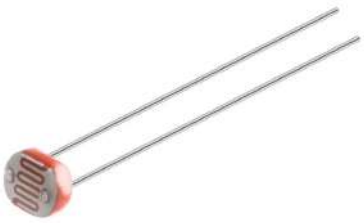
For real-time visual feedback, an LCD 2004 module is integrated. To optimize the limited GPIO resources of the ESP32-CAM, the display is equipped with a PCF8574 I2C backpack. This adapter converts parallel data into I2C serial communication, requiring only two data lines (SDA and SCL) to display system statuses.



Figure 2.6: LCD 2004 with PCF8574 I2C Interface Backpack.

2.3.6 Light Dependent Resistor (LDR) and Illumination LED

To ensure reliable operation in low-light environments, a Light Dependent Resistor (LDR) is employed to continuously monitor environmental lux levels. When the ambient light falls below a predefined threshold, the system automatically triggers a high-intensity White LED to illuminate the user's face.



(a) Light Dependent Resistor (LDR).



(b) High-Intensity White LED.

Figure 2.7: Low-Light Compensation Components.

2.4 Hardware Design

2.4.1 Schematic Design

The schematic diagram was meticulously designed to ensure signal integrity and power isolation for the system. Based on the schematic, the circuit is divided into the following core subsystems:

- **Power Distribution:** A 9V DC input from `9V_Jack1` is supplied directly to the relay contact to drive the high-power load (solenoid lock). Simultaneously, the `LM2596S1` DC-DC Buck module steps down this 9V input to a stable 5V DC level for the ESP32-CAM microcontroller. The power filtering network utilizes a combination of a 470μF electrolytic capacitor (for bulk decoupling and transient current compensation) and a 100nF ceramic capacitor (for high-frequency noise suppression), preventing the MCU from resetting due to sudden voltage drops when the magnetic lock activates.
- **Actuation & Isolation:** To drive the 5V relay (`SRD-05VDC-SL-C1`) using the 3.3V logic signal from the ESP32, an NPN BJT (`2N2222A1`) is implemented as a low-side switch via a 1kΩ base resistor (R_4). Crucially, a `1N4007` flyback diode

(D_5) is placed in reverse parallel with the relay coil. This configuration completely suppresses the inductive kickback (flyback current), strictly protecting the transistor's collector terminal from breakdown voltage.

- **Sensing & Indication:** The `LDR03` photoresistor, combined with a 10kΩ pull-up resistor (R_2), forms a voltage divider that sends an analog signal to pin `IO13` to measure ambient illuminance. Alert and indication functions are executed via a white illumination LED (D_3, pin `IO4`) and an active buzzer (BZ_1, pin `IO2`), utilizing 220Ω current-limiting resistors (R_3, R_6) to protect the MCU's I/O ports.
- **Communication:** Header `J1` facilitates the connection to the LCD 1602 module. Implementing the I2C communication bus requires only two signal pins, `IO14` (SCL) and `IO15` (SDA), instead of traditional parallel wiring, thereby profoundly optimizing the limited I/O resources on the ESP32-CAM.

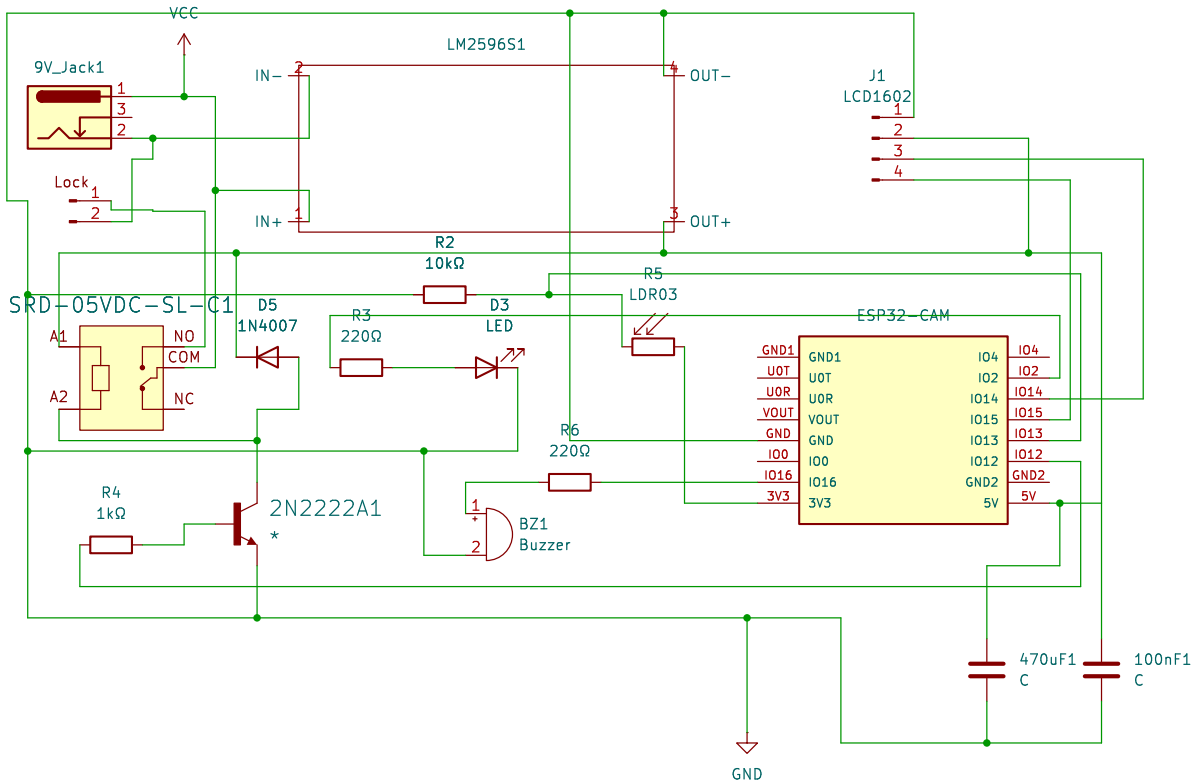


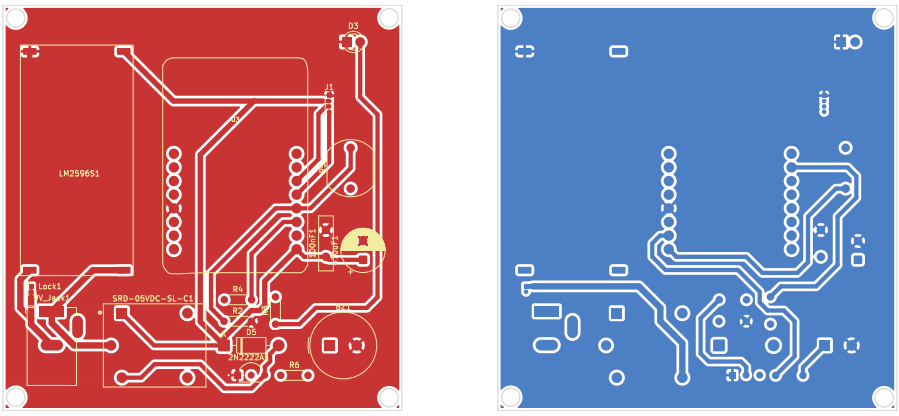
Figure 2.8: Overall System Schematic Diagram.

2.4.2 Printed Circuit Board (PCB) Design

The Printed Circuit Board (PCB) layout was engineered as a double-sided board using KiCad EDA, strictly adhering to electromagnetic compatibility (EMC) standards and optimal physical floorplanning:

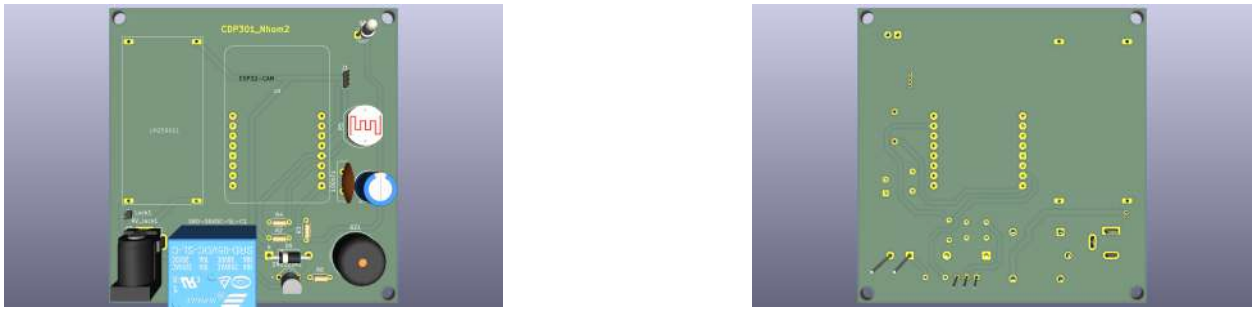
- **Floorplanning:** The board layout demonstrates a distinct physical boundary between the low-voltage signal domain and the high-power domain. The LM2596 buck converter and the electromechanical relay are clustered near the board's edge, thoroughly isolated from the ESP32-CAM "brain" in the center. This placement prevents radiated magnetic fields from the LM2596 inductor and the relay coil from coupling into the Wi-Fi antenna and sensitive signal traces.
- **Routing & Trace Width:** Trace widths were optimally calculated based on current-carrying requirements. The power nets (VCC 9V, 5V, and GND) utilize wide traces to minimize pure resistance, eliminate IR drops, and dissipate heat efficiently. Conversely, digital logic traces (I2C and GPIO) are routed with narrower widths using the shortest possible paths, incorporating 45° mitered corners to minimize signal reflection and charge accumulation.
- **GND Copper Pour:** Observing the bottom layer, the entire empty routing space is filled with a grounded copper pour, featuring thermal reliefs at component pads. This continuous copper plane exhibits extremely low impedance, functioning as a Faraday shield to protect the circuit from crosstalk while providing an optimal, short return path for digital signals.
- **Mechanical Support:** The four corners of the PCB accommodate mounting holes surrounded by safe copper keepout zones. This structural design ensures no short circuits occur when securing the board with metal screws to the door enclosure, satisfying the physical vibration constraints of real-world operation.

To provide a comprehensive view of the manufacturing output, 3D models of both the top and bottom sides are generated. These 3D perspectives allow the design team to visually evaluate the ergonomics, safety clearances, and physical constraints before exporting the Gerber files.



(a) Top view. (b) Bottom view.

Figure 2.9: Component Placement and Routing (2D PCB Layout).



(a) Top View. (b) Bottom View.

Figure 2.10: 3D PCB Model Perspectives.

2.5 Software Design and Algorithms

2.5.1 System Operation Flowchart

The firmware follows an event-driven loop to ensure real-time responsiveness, categorized into four execution phases:

- **Initialization:** Upon `Start`, the system initializes all peripherals, including the camera, GPIO pins, I2C LCD, and local Wi-Fi connectivity.
- **Standby & Triggering:** The system continuously monitors the photoresistor (`Read LDR`). Upon detecting a trigger (`Is LDR Covered? - Yes`), it immediately exits the standby mode.
- **Acquisition & Transmission:** The system activates the illumination LED and captures an image (`Capture Image and Flash LED`). This data is then transmitted to the AI server for feature extraction (`Send to AI Server`).

- **Authentication & Actuation:** Based on the server's response (`Face Recognized?`):
  - **Denial (No):** LCD displays `"Access Denied"` and returns to standby.
  - **Approval (Yes):** LCD displays `"Access Granted"`, the relay energizes (`Activate 5V Relay`) for a 3-second window, and then resets (`Deactivate 5V Relay`) to standby.

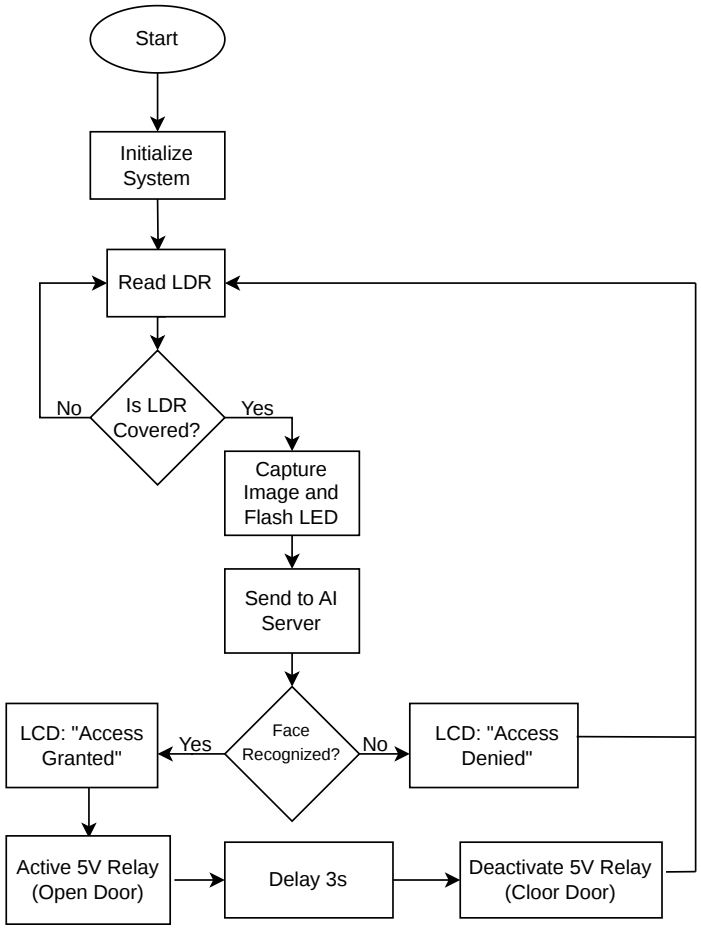


Figure 2.11: System Firmware and Operation Flowchart.

2.5.2 Facial Recognition Algorithm

The facial recognition subsystem does not execute on the ESP32-CAM itself. As established in Section 2.5.1, the microcontroller's sole responsibility is image acquisition, low-light compensation, and transmission (`Send to AI Server`); the computationally intensive work of detecting, encoding, and verifying a face

is delegated to a remote AI server integrating the Face++ (Megvii Inc.) cloud recognition engine [1][4]. Because Megvii does not disclose the exact production network topology or training recipe behind its commercial API, the description below reflects the general deep-learning architecture pattern established in the face-recognition literature – a pattern corroborated both by Megvii's own published research on its recognition engine and by the observable behaviour of the public API. The pipeline consists of four sequential stages: face detection, landmark localisation and alignment, deep feature extraction, and similarity-based matching.

Face Detection and Landmark Alignment Given a frame I captured by the ESP32-CAM, the detector must first localise the face region before any recognition can occur. Deep, cascade-style detectors of the class underlying commercial engines jointly regress a bounding box and classify each candidate region as *face* or *non-face*. For a candidate region, the face-confidence score is obtained by passing the convolutional features $\phi(I)$ through a classification head and a sigmoid activation:

$$p_{face} = \sigma\left(w_c^T \phi(I) + b_c\right) = \frac{1}{1 + e^{-(w_c^T \phi(I) + b_c)}} \tag{1}$$

where $\phi(I)$ is the deep feature map produced by the convolutional layers and w_c, b_c are the learned weight vector and bias of the classifier. A region is retained only if p_{face} exceeds a detection threshold (typically 0.90–0.95).

In parallel, a regression head predicts the offset between a candidate anchor box (x_a, y_a, w_a, h_a) and the true face box (x, y, w, h):

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log\frac{w}{w_a}, \quad t_h = \log\frac{h}{h_a} \tag{2}$$

Because several overlapping anchors typically fire on one face, redundant boxes are pruned via Non-Maximum Suppression, governed by the Intersection-over-Union metric:

[4]Megvii Technology, *Face++ Cognitive Services API Documentation*, 2015.

$$IoU(B_1, B_2) = \frac{|B_1 \cap B_2|}{|B_1 \cup B_2|} \tag{3}$$

Boxes whose IoU with a higher-confidence detection exceeds approximately 0.5 are discarded, leaving one bounding box per face.

Once isolated, the face is passed through landmark localisation. Face++'s standard Landmark service returns 83 or 106 keypoints covering the face contour, eyes, eyebrows, nose, and lips, with a Dense Landmarks variant extending this to 1,000 points for higher-precision contour mapping – a capability with an established track record, having won the 300 Faces In-the-Wild landmark detection challenge in 2013. These points $\{(x_i, y_i)\}_{i=1}^{N}$ are used to compute a similarity transform that rotates, scales, and translates the detected face into a canonical frontal pose, by minimising the squared error against a reference template $\{(x'_i, y'_i)\}$:

$$\min_{s,\theta,\mathbf{t}} \sum_{i=1}^{N} \left\| \begin{bmatrix} x'_i \\ y'_i \end{bmatrix} - \left(sR(\theta) \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \mathbf{t} \right) \right\|^2 \tag{4}$$

where s is a uniform scale factor, $R(\theta)$ the 2-D rotation matrix, and $\mathbf{t}$ the translation vector. This alignment step matters because the downstream embedding network is trained almost exclusively on frontalised faces; an unaligned input measurably degrades verification accuracy.

Deep Feature Extraction (Facial Embedding) The aligned crop $I_{aligned}$ is passed through a deep CNN backbone $f_\theta(\cdot)$ that maps the image to a fixed-length embedding:

$$\mathbf{v} = f_\theta(I_{aligned}) \in \mathbb{R}^d \tag{5}$$

with d typically 128–512, followed by L_2 normalisation so the embedding lies on a unit hypersphere:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \tag{6}$$

The embedding's usefulness – placing the same identity close together while pushing different identities apart – comes entirely from the training objective. Rather than a plain classification loss, state-of-the-art engines in this class are trained with a margin-based softmax objective, the Additive Angular Margin Loss, or ArcFace [2][5]:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{s\cdot\cos(\theta_{y_i}+m)}}{e^{s\cdot\cos(\theta_{y_i}+m)}+\sum_{j=1,\,j\neq y_i}^{C}e^{s\cdot\cos\theta_j}} \tag{7}$$

Each term in Equation 7 carries a specific meaning, summarised in Table 2.1.

Table 2.1: Definition of terms in the ArcFace loss function (Eq. 7).

Symbol	Meaning
N	Number of samples in a training mini-batch
C	Number of distinct identities in the training set
θ_{y_i}	Angle between sample i's embedding $\hat{\mathbf{v}}_i$ and its true-identity weight vector W_{y_i}, i.e. $\cos\theta_{y_i} = \hat{\mathbf{v}}_i^{\top}\hat{W}_{y_i}$
θ_j	Angle between the embedding and every other (incorrect) identity j
m	Additive angular margin (commonly ≈ 0.5 rad), added to the correct class's angle to make it deliberately harder to satisfy during training
s	Scale factor (commonly $s = 64$) that stretches bounded cosine values ($[-1,1]$) into a range the softmax can train on effectively

Geometrically, this loss compresses same-identity embeddings into a tight angular cluster while enforcing a minimum angular gap m between different identities' clusters – precisely the separation the matching stage below depends on.

Face Matching and Access Decision At inference time there is no training – only a comparison between the newly captured embedding $\hat{\mathbf{v}}_{probe}$ and the enrolled

[5] J. Deng et al., "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *CVPR*, 2019.

users' stored embeddings $\hat{\mathbf{v}}_k$. Since both are unit vectors, similarity reduces to a dot product:

$$\text{sim}(\hat{\mathbf{v}}_{probe}, \hat{\mathbf{v}}_k) = \hat{\mathbf{v}}_{probe}^{\top} \hat{\mathbf{v}}_k = \cos\theta_k \tag{8}$$

For unit vectors this is mathematically equivalent to Euclidean distance, since $\|\hat{\mathbf{v}}_{probe} - \hat{\mathbf{v}}_k\|_2^2 = 2 - 2\cos\theta_k$: maximising similarity and minimising distance yield the same decision.

Rather than exposing a raw cosine value, Face++'s Compare API reports a confidence score from 0 to 100, where higher values indicate greater similarity between the two faces, and recommends different cutoffs by use case: 60–70 for low-friction personalisation, 70–80 for standard authentication such as app login, and 80 or above for high-security scenarios such as financial transactions or border control. Because a physical door lock sits firmly in the high-security category, the project adopts a threshold at the upper end of this range to bias the system toward a low False Acceptance Rate (FAR) – an intruder wrongly admitted – even at the cost of a marginally higher False Rejection Rate (FRR) – a legitimate user occasionally re-presenting their face:

$$\text{Decision} = \begin{cases} \text{Access Granted}, & \text{confidence} \geq \tau \\ \text{Access Denied}, & \text{confidence} < \tau \end{cases} \tag{9}$$

This is exactly the `Face Recognized?` branch of the system flowchart in Figure 2.11. The core verification logic executed on the AI server can be summarised as:

Listing 1: Face verification logic on the AI server.

```
import requests

COMPARE_URL = "https://api-us.faceplusplus.com/facepp/v3/compare"
API_KEY, API_SECRET = "<api_key>", "<api_secret>"
THRESHOLD = 80 # high-security tier, per Face++ guidance

def verify_face(captured_b64, enrolled_b64):
```

```
    resp = requests.post(COMPARE_URL, data={
        "api_key": API_KEY,
        "api_secret": API_SECRET,
        "image_base64_1": captured_b64,
        "image_base64_2": enrolled_b64,
    }).json()
    confidence = resp.get("confidence", 0)
    return confidence >= THRESHOLD, confidence
```

If `verify_face` returns `True` against any enrolled user, the server signals the ESP32-CAM to display `"Access Granted"` and energise the relay; otherwise `"Access Denied"` is shown and the door remains locked (Section 2.5.1).

Accuracy and Algorithm Selection Rationale Megvii's own published research (Zhou *et al.*, 2015) reports its Face Recognition System reaching 99.50% verification accuracy on the LFW (Labeled Faces in the Wild) benchmark, the standard unconstrained face-verification dataset [3][6]. This places it within the same accuracy tier as other leading deep-learning systems: Facebook's DeepFace reached 97.35% on LFW in 2014 [4][7], and Google's FaceNet reached 99.63% the following year after training on roughly 200 million images of about 8 million identities using a triplet-loss objective, while ArcFace later raised the figure most often cited as the current state of the art to 99.83%. Table 2.2 summarises these results alongside classical, non-deep-learning approaches.

The "highly condition-dependent" qualifier for the classical row is not a hedge – it reflects genuine disagreement in the literature. One comparative study measured Eigenfaces, Fisherfaces, and LBPH at just 40%, 53.3%, and 79% accuracy respectively, while another, testing the same three algorithms under varied weather conditions, found Fisherfaces reaching 100% and LBPH as the weakest performer at 98%. Because these methods rely on hand-crafted, low-dimensional pixel representations rather than learned features, their accuracy swings by tens of percentage points depending on lighting, pose, and dataset – precisely the illumination vari-

[6] G. B. Huang et al., "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," 2007.

[7] Y. Taigman et al., "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," *CVPR*, 2014.

Table 2.2: Comparison of face recognition algorithm families on the LFW benchmark.

Category	**Method**	**Core Technique**	**Typical LFW Accuracy**
Classical / holistic	Eigenfaces (PCA)	Linear variance-maximising projection	Highly condition-dependent
Classical / holistic	Fisherfaces (LDA)	Linear discriminant projection	Highly condition-dependent
Classical / local texture	LBPH	Local binary pattern histograms	Highly condition-dependent
Deep learning	DeepFace (2014)	9-layer CNN + 3-D alignment	97.35%
Deep learning	FaceNet (2015)	CNN + triplet-loss embedding	99.63%
Deep learning	Megvii / Face++	Large-scale CNN embedding	99.50%
Deep learning	ArcFace (2019)	CNN + additive angular margin	99.83%

ability this project explicitly engineers around in Section 2.3.6. That variability alone makes them unsuitable for a lock that must operate reliably at any hour.

Reaching the accuracy of a modern deep embedding network above 99.5%, however, normally requires either training a large CNN from scratch on hundreds of millions of face images – infeasible for a semester-length capstone – or running a pretrained model such as ArcFace locally, which demands several hundred megabytes of RAM and GPU-class throughput per inference. Neither option is compatible with the ESP32-CAM, whose entire compute budget (Section 2.3.1) is a 240MHz dual-core microcontroller with a few hundred kilobytes of usable RAM and no dedicated neural accelerator. Face++ resolves this conflict by hosting detection, alignment, embedding, and matching entirely on Megvii's servers: the embedded hardware's only job is capturing a JPEG frame and transmitting it over Wi-Fi (Section 2.2), while the accuracy above 99.5% is delivered remotely. This division of labour lets the project reach state-of-the-art recognition accuracy without imposing memory, power, or cost constraints on the embedded side, while the tunable confidence threshold (Equation 9) gives the team direct, quantified control over the FAR/FRR trade-off appropriate to a physical door lock. A natural extension for future work would be enabling Face++'s liveness-detection module to additionally guard against photograph-based spoofing, which the current prototype does not yet implement.

3. SYSTEM TESTING AND RESULTS

3.1 Real-Life Deployment

Figure 3.1 illustrates the hardware prototype of the face recognition door lock system. As a proof-of-concept, the setup was constructed using cardboard to simulate a physical door and frame. The electronic components, including the display, microcontroller, and camera module, are securely mounted on the exterior of the model. This setup effectively demonstrates the system's practical operation, spatial arrangement, and user interaction within a laboratory environment.

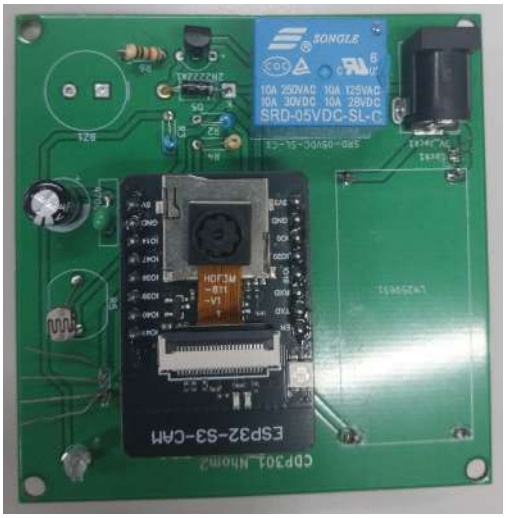


Figure 3.1: Prototype Deployment of the Face Recognition Door Lock System.

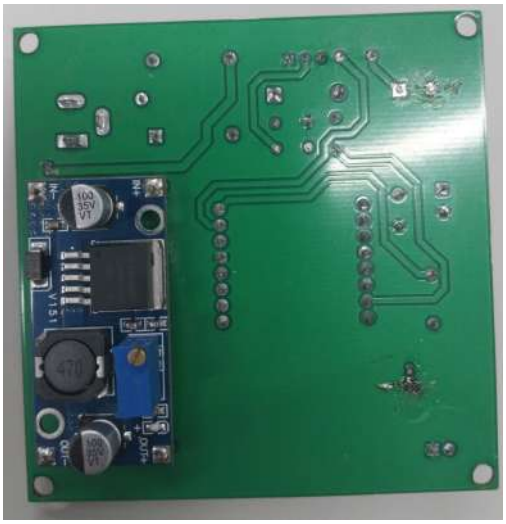
3.2 Hardware Prototype

The hardware implementation centers on a custom-designed PCB that serves as the integration hub for the system's primary components. The ESP32-CAM module, responsible for image acquisition and local processing, is securely mounted onto the PCB, which provides regulated power management through an onboard DC-DC step-down converter, ensuring stable operation for both the 5V logic and the 12V solenoid lock. The system's electromechanical interface is managed by a high-current relay, enabling the ESP32-CAM to actuate the solenoid lock seamlessly

upon successful authentication. While the integration into a final enclosure is currently in the conceptual design phase, all core electronic modules—including the LCD interface, LDR-based lighting trigger, and solenoid locking mechanism—have been fully assembled and verified as functional, meeting all operational requirements for the proposed intelligent access control system.



(a) Front side of assembly PCB.



(b) Back side of assembly PCB.

Figure 3.2: Assembled PCB.

3.3 Functional Hardware Verification

The physical response of the hardware was strictly verified through sequential state transitions. Figures 3.3, 3.4, and 3.5 provide a visual framework...

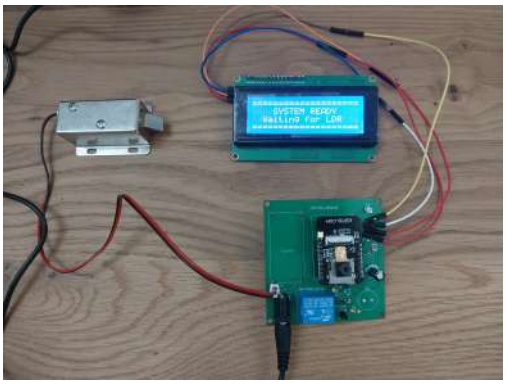


Figure 3.3: Test Case 1: Initial Standby State.

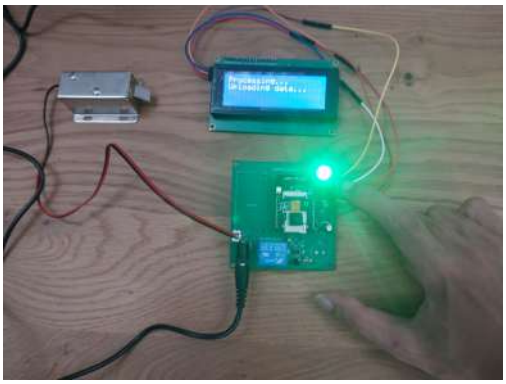


Figure 3.4: Test Case 2: LDR Covered for Image Capture (LED On).

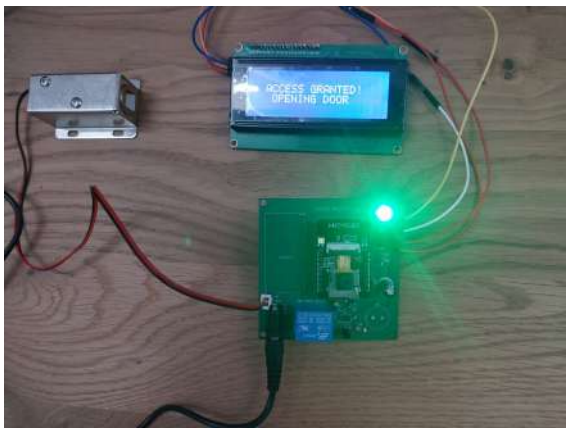


Figure 3.5: Test Case 3: Solenoid Retracted (Unlocked).

The functional logic operates rigorously through the following test cases:

- **Test Case 1 - Initial Standby State (Figure 3.3):** The system rests in a low-power monitoring mode. The solenoid lock remains fully extended, maintaining a Fail-Secure state to mechanically lock the door.
- **Test Case 2 - LDR Triggering and Image Capture (Figure 3.4):** Upon intentionally covering the Light Dependent Resistor (LDR), the ambient light threshold drops significantly. The ESP32-CAM detects this transition and immediately triggers the high-intensity white LED for low-light compensation before capturing the facial image.
- **Test Case 3 - Actuation and Unlocking (Figure 3.5):** Following a successful API verification from the server, a distinct mechanical "click" is produced as the 5V relay energizes. The relay completes the 12V high-power circuit, magneti-

cally pulling the solenoid slug inward to unlock the door for the predefined delay period.

3.3.1 Web-Based Monitoring and History Log

To provide transparency and enable systematic performance auditing, the system architecture includes a web-based dashboard that logs every facial recognition event in real-time. This interface captures critical metadata, including authentication status, confidence scores, processing latency, and temporal markers. Figure **??** presents a representative sample of these historical logs, showcasing both successful identification events and edge-case failures, which serve as the primary dataset for the quantitative analysis detailed in the following section.



Figure 3.6: Log Page 1.



Figure 3.7: Log Page 2.

3.4 Experimental Results and Accuracy

To quantitatively assess the facial recognition performance via the cloud API, a series of 62 experimental trials were conducted. The dataset comprised both registered individuals (authorized) and unregistered individuals (strangers).

Table 3.1 summarizes the recognition accuracy over 62 trials.

Test Category	**Total**	**Success**	**Acc. (%)**
Authorized Users	28	27	96.4%
Unauthorized Users	23	20	87.0%
Overall	**51**	**47**	**92.2%**

Table 3.1: Summary of Facial Recognition Experimental Results.

*Note: 11 trials were excluded from the accuracy calculation due to 'No face' detection errors; successful identification was recorded in low-light conditions with the aid of the integrated LED.

3.4.1 Limitations and Analysis

The facial recognition model demonstrated robust performance under normal conditions; however, specific edge cases highlighted vulnerabilities in the system's current configuration:

- **Lookalike Vulnerability & Accessory Impact:** The system's sensitivity was rigorously tested using individuals wearing glasses (18 trials). A critical

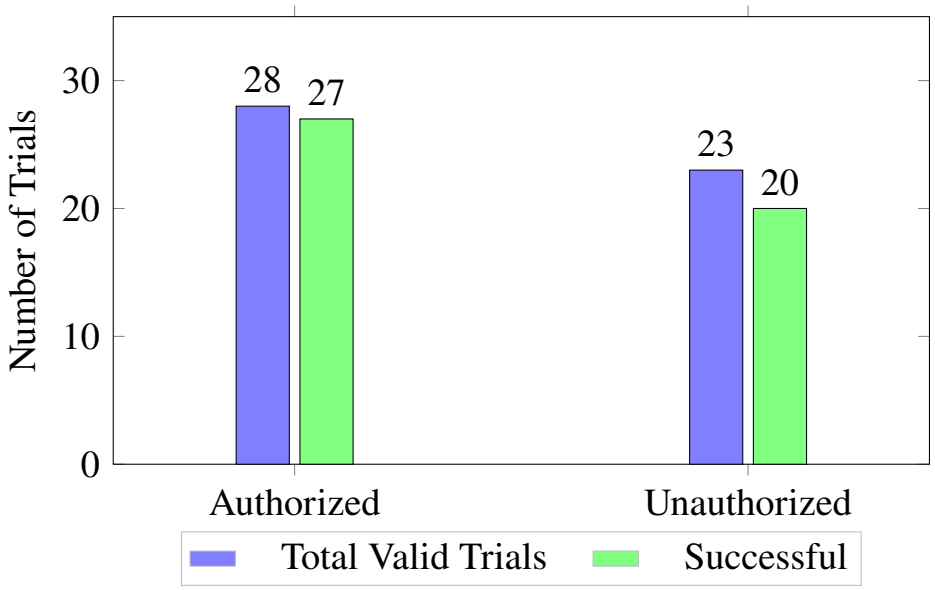


Figure 3.8: Comparison of Total vs. Successful Identifications.

edge case emerged involving an unregistered individual who shared a similar skin tone and hairstyle with a registered team member. Out of 8 trials with this specific individual:

  - While wearing glasses, the system recorded 3 False Acceptances and only 1 correct rejection.
  - When the glasses were removed or the hairstyle was altered, the system successfully rejected the individual (4 trials). However, the generated confidence scores remained unusually high (ranging from 70% to 74.8%).

- **Intra-class Misidentification:** In addition to the lookalike issue, 1 instance was recorded where a registered user was incorrectly identified as another registered user due to severe facial occlusion.
- **Image Orientation:** The system exhibited sensitivity to non-standard orientations. In tests with 180-degree inverted images, the system achieved a 75% success rate (3 out of 4 trials).
- **Environmental Sensitivity:** A total of 11 trials (approx. 18%) initially failed due to 'No face' detection, primarily in extreme low-light environments. However, the system's integrated high-intensity white LED significantly

improved performance in suboptimal lighting. In several instances where ambient light was near zero, the hardware successfully triggered the LED compensation, enabling the cloud API to achieve identification with confidence scores reaching up to 87.8%, proving the efficacy of the current hardware-based lighting solution.

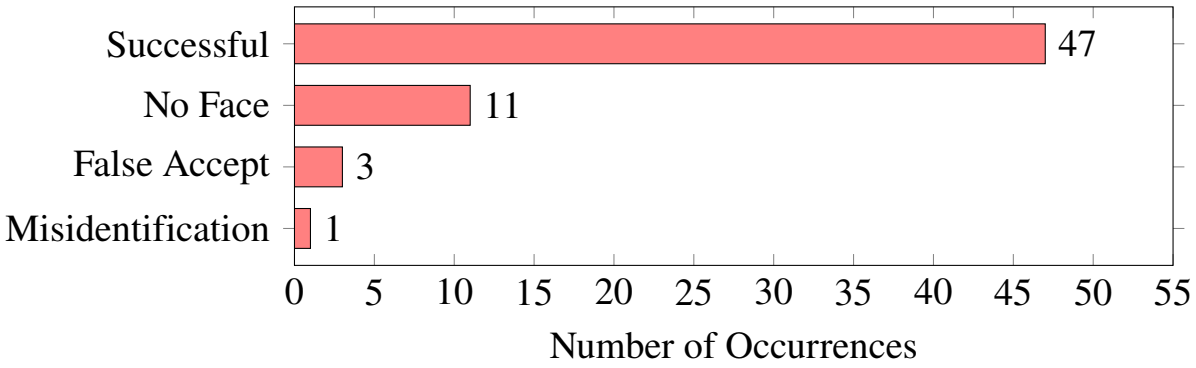


Figure 3.9: Breakdown of System Outcomes across 62 Total Trials.

3.4.2 Future Improvements

To address these limitations, future iterations will focus on:

- **Threshold Optimization & Liveness Detection:** Re-evaluating the 75% confidence threshold and potentially implementing liveness detection to mitigate the lookalike and accessory-driven False Acceptance risks.
- **Feature Extraction Enhancement:** Training the model to rely more heavily on invariable biometric features rather than easily altered attributes like hairstyles or glasses.
- **Image Normalization:** Adding an automated image orientation correction algorithm on the ESP32-CAM before transmitting the payload to the cloud API.

3.4.3 API Latency Analysis and Network Jitter

Since the core facial recognition inference is processed on a cloud server via an API, evaluating end-to-end network latency is critical. Real-world testing revealed fluctuating processing times depending on network stability.

Under optimal network conditions, the end-to-end processing latency is maintained at approximately 1950ms. Table 3.2 quantifies the average execution time breakdown across successful authentication cycles. Empirical data indicates that the primary bottleneck lies within the image payload transmission (HTTP POST) over the local WiFi network.

Processing Phases	**Average Time (ms)**	**Proportion (%)**
1. Image Capture & JPEG Compression	200	10.3%
2. HTTP POST Request (Network Upload)	1350	69.2%
3. Cloud API Inference (Server-side)	350	17.9%
4. Response Parsing & Actuation	50	2.6%
Total Optimal Latency	**1950**	**100%**

Table 3.2: End-to-End Latency Breakdown Under Optimal Network Conditions.

However, during periods of network congestion or weak signal to the ESP32-CAM, the total latency occasionally spiked to 2.5 – 5.0 seconds. This variance is primarily attributed to TCP packet retransmissions when the HTTP request drops.

3.4.4 System Reliability and Asynchronous Execution Observations

Real-world continuous integration testing highlighted specific behaviors in system synchronization and error handling:

- **Frontend Synchronization Delay:** An asynchronous anomaly was observed between the hardware actuation and the Web Dashboard update. In certain instances, the ESP32-CAM successfully received the authentication token and triggered the solenoid relay instantly; however, the web interface experienced a severe delay in logging the event. The dashboard occasionally required a subsequent capture event to force a data refresh. This indicates a bottleneck in the frontend polling mechanism (or WebSocket connection) rather than the backend facial recognition process.
- **Network Timeout Handling:** If the API fails to respond within a predefined threshold (e.g., 5000ms), the ESP32-CAM terminates the request, displays "Network Error" on the LCD, and triggers a continuous error tone via the buzzer to prevent the system from hanging indefinitely.

- **Invalid Payload Response:** If the captured image is corrupted during transmission, the server returns an HTTP 400 Bad Request status, prompting the hardware to silently reset the capture cycle without actuating the lock.

4. CONCLUSION

4.1 Summary of Achievements

This project successfully designed and implemented a Face Recognition Door Lock System that achieved the objectives defined in Section 1.1.1. The hardware architecture ensured safe isolation between the ESP32-CAM and the 12V solenoid lock, while the software integrated the Face++ API for biometric authentication. Experimental results demonstrated reliable recognition of authorized users with an average end-to-end latency of approximately 1.8–2.1 seconds. The prototype also showed stable operation during repeated testing, confirming its feasibility as a practical smart access control solution for homes, laboratories, and small offices.

4.2 System Limitations

Although the prototype operated successfully, several limitations remain. The system depends on the Face++ cloud API and a stable Wi-Fi connection, making it vulnerable to network interruptions and increased response time. In addition, recognition performance may decrease under poor lighting, extreme viewing angles, or when the user's face is partially occluded. Furthermore, the current prototype has not been evaluated with a large number of users, so its long-term accuracy and scalability require further validation.

4.3 Future Works

Future improvements include replacing the cloud-based API with an offline edge-AI model to reduce latency and remove internet dependency. Integrating liveness detection would enhance security against spoofing attacks. Furthermore, developing a companion mobile application would enable user management, real-time intrusion notifications, and access history monitoring. Additional optimization of the recognition algorithm and hardware resources could further improve system performance, reliability, and power efficiency for long-term deployment.

References

[1] Megvii Technology, *Face++ Cognitive Services API Documentation*, 2015. [Online]. Available: https://www.faceplusplus.com/

[2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690-4699.

[3] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," *University of Massachusetts, Amherst, Tech. Rep. 07-49*, October 2007.

[4] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1701-1708.

[5] M. A. Al-Qurabat and A. K. Idrees, "Smart Home Security System Based on Internet of Things," *2020 1st Information Technology To Enhance e-learning and Other Application (IT-ELA)*, Baghdad, Iraq, 2020, pp. 1-6.

[6] A. K. Gupta, et al., "Smart Door Lock System using ESP32 CAM," *International Journal of Engineering Research & Technology (IJERT)*, vol. 10, no. 5, pp. 450-454, May 2021.

[7] K. K. Patel and S. M. Patel, "Internet of Things-IOT: Definition, Characteristics, Architecture, Enabling Technologies, Application & Future Challenges," *International Journal of Engineering Science and Computing*, vol. 6, no. 5, pp. 6122-6131, 2016.

APPENDIX

A. Wiring Connection and Pinout Table

To assist beginners and developers in reproducing this project, Table A.1 details the specific pin connections between the ESP32-CAM, the external components, and the power module.

Table A.1: ESP32-CAM Pinout and Connections.

Component	**ESP32-CAM Pin**	**External Module Pin**	**Note**
I2C LCD 2004	GPIO 14	SDA	Requires 5V VCC
I2C LCD 2004	GPIO 15	SCL	
Relay Module	GPIO 12	2N2222A Transistor	Base pin of 2N2222A to trigger relay
LDR Sensor	GPIO 13	DO (Digital)	Detects ambient light
LED (Flashlight)	GPIO 2	IN / Anode	Triggered in low light
Power Supply	5V / VCC	LM2596 Output	Buck converter from 12V
Power Supply	GND	Common GND	Ensure all grounds are connected

B. Environment Setup and Flashing Firmware

To simplify the firmware uploading process, this project utilizes the **ESP32-CAM MB** programmer shield, which eliminates the need for complex manual wiring with standard FTDI USB-to-TTL modules. Users can follow these steps to flash the firmware:

1. **Hardware Assembly:** Attach the ESP32-CAM onto the ESP32-CAM MB shield and connect it to the computer via a micro-USB cable.
2. **IDE Setup:** Open Arduino IDE, ensure the ESP32 board manager is installed, and select **AI Thinker ESP32-CAM** from the Boards menu.
3. **Flashing Procedure:** Click the "Upload" button in the Arduino IDE. When the IDE console displays the "Connecting..." or uploading text, press and hold the **IO0** button on the MB shield, then press the **RESET** button on the ESP32-CAM board. This sequence forces the chip into download/flash mode. Release the buttons once the flashing process begins.

C. ESP32-CAM Source Code (Firmware Snippet)

Below is a core snippet of the ESP32-CAM firmware written in C++ (Arduino framework) demonstrating the camera capture and network logic.

Listing 2: ESP32-CAM Capture logic snippet.

```
void initCamera() {
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;

  config.xclk_freq_hz = 16000000;
  config.pixel_format = PIXFORMAT_JPEG;
  config.frame_size = FRAMESIZE_VGA;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  config.fb_location = CAMERA_FB_IN_DRAM;

  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Init Camera failed: 0x%x\n", err);
    lcd.setCursor(0, 2);
```

```
    lcd.print("Camera: Error      ");
  } else {
    Serial.println("Camera Init OK");

    sensor_t * s = esp_camera_sensor_get();
    if (s) {
      s->set_framesize(s, FRAMESIZE_VGA);
    }
    lcd.setCursor(0, 2);
    lcd.print("Camera: OK         ");
  }
}
```

D. Project Repository (Open Source)

For the complete source code (including the ESP32-CAM firmware, Python AI Server backend, and API integration), as well as the PCB Gerber files and full schematics, please visit our open-source repository at GitHub:

GitHub Repository Link:
`https://github.com/trangiaphuc05022006-eng/ESP32-CAM-source-code`

This repository is publicly available to help hobbyists, students, and researchers easily clone, build, and modify the system.